

ATH-38 Review — Calendar Agent

What was built

- lib/agents/calendar-agent.ts — LangGraph node using structured output
 - lib/agents/prompts/calendar-draft.md — system prompt
 - lib/agents/__tests__/calendar-agent.test.ts — test suite
-

● Blockers

1. Tests hit real OpenAI — no mocks at all

// No vi.mock() anywhere

const result = await calendarAgent(state) // calls real gpt-4o

Both tests make live LLM calls. They require OPENAI_API_KEY in the environment, cost money on every test run, and are non-deterministic — the LLM can return a slightly different summary wording and break the toMatch(/Alice/i) assertion on any given run. Neither the spec's required scenario ("meeting with Alice tomorrow 2pm for 1h") nor the 30-min case will pass reliably in CI. model needs to be mocked exactly like Ahaon's report-agent.test.ts does it:

```
vi.mock("../langgraph/llm-factory", () => ({
  model: { withStructuredOutput: vi.fn(() => ({ invoke: vi.fn() })) }
}))
```

2. __dirname is not available in Next.js (ESM)

const promptPath = path.join(__dirname, "prompts", "calendar-draft.md")

__dirname is a CommonJS global. Next.js compiles to ESM where it's undefined — this throws ReferenceError: __dirname is not defined the moment the agent is called in any Next.js route or LangGraph graph. The report-agent uses the correct pattern:

path.join(process.cwd(), "lib/agents/prompts/calendar-draft.md")

3. pending_action and awaiting_approval are not in the state schema

```
return {
  awaiting_approval: true,
  pending_action: { type: 'calendar-create', payload: draft },
}
```

AtheneStateType has: messages, orgId, userId, role, next, retrievedDocs, final_answer. It has no pending_action or awaiting_approval fields. LangGraph silently drops state keys it doesn't recognise — the HITL interrupt will never actually gate on anything. These fields need to be added to the state annotation before this agent can work end-to-end.

4. state.user?.timezone path doesn't exist

const userTimezone = state.user?.timezone || "UTC"

AtheneStateType has no user object. This silently falls back to UTC on every call, meaning the timezone handling in the prompt is always wrong for any non-UTC user. The timezone either needs to be added to AtheneStateType as a top-level field (timezone?: string) or the agent should accept it as a config parameter.

● Should fix

5. state: any — no type safety

The function signature `calendarAgent(state: any)` means TypeScript won't catch any state field mismatches. Should be `state: AtheneStateType` (and fix issues 3 & 4 in the process). Every other agent in the codebase uses the typed state.

6. Error return is inconsistent with LangGraph conventions

```
return { error: "Could not parse calendar event details." }
```

error is not a state field either. On LLM parse failure the agent should push a message into `state.messages` so the graph can route or surface it:

```
return {  
  messages: [{ role: "assistant", content: "Could not parse event details, please rephrase." }]  
}
```

7. Prompt is very thin for the scope of this agent

`calendar-draft.md` is 7 lines. It handles relative times and defaults but has no guidance for: ambiguous times ("morning", "afternoon"), multi-day events, missing attendee emails (user says "Alice" not "alice@company.com"), or the fact the agent is drafting only and should not claim it's creating anything. Compare to `cross-dept.md` which explicitly lists what the agent must NOT do. Worth expanding before it goes to production.

● Minor / informational

8. Prompt re-read on every invocation

`fs.readFileSync` is called on every agent invocation, not once at module load. Move it to module scope (with `process.cwd()` fix above) so it's read once.

9. Attendee schema is email-only

`attendees: [{ email }]` — both Google Calendar and MS Graph accept `{ email, displayName }`. Not blocking but the LLM will extract names that then get discarded.

10. chrono-node not used

Spec mentioned it as an option — LLM-only parsing is fine, just not documented anywhere.

Summary

Item	Severity
Tests make live OpenAI calls — no mocks	● Blocker
<code>__dirname</code> throws in ESM/Next.js	● Blocker
<code>pending_action</code> / <code>awaiting_approval</code> not in state schema	● Blocker
<code>state.user?.timezone</code> path doesn't exist → always UTC	● Blocker
<code>state: any</code> — untyped	● Should fix
Error return not in state schema	● Should fix

Prompt too thin for edge cases

● Should fix

fs.readFileSync on every call

● Minor

Four blockers — the agent cannot run correctly as-is and the tests will never pass in CI. The core approach (structured output via Zod schema, HITL pattern, prompt + context injection) is the right direction. The fixes are all mechanical: mock the LLM in tests, swap `__dirname` for `process.cwd()`, add `pending_action/awaiting_approval/timezone` to the state annotation, and type the function signature properly.

1.